

Proving a goal without exposing the person

How Mitra verifies that someone did what they said they would, and why it publishes a sentence instead of a document.

Decided 2026-09-19. Supersedes the redaction design of the same day. Decision records: D06 and D07 in DECISIONS.md.

1. The problem

Mitra is a play-money prediction market about people's real goals. Someone writes a goal about themselves — finish the semester with a 3.8, land the internship, deadlift 315 — the owner approves it, and their friends trade YES or NO on whether it happens.

For any of that to mean anything, the market has to resolve on what actually happened. Which means somebody has to prove it.

That is where the app meets a problem that has nothing to do with markets.

The proof is the most sensitive thing the app will ever touch. A GPA goal resolves on a transcript. A transcript carries a student's name, their student ID number, their date of birth, their home address, their phone number, and every grade they have ever received. An internship goal resolves on an offer letter, which carries a salary and a home address. The proof and the private data arrive in the same file, and you cannot have one without the other.

So the app has three things it must do at once, and they pull against each other:

It must	Because
Establish what actually happened	Otherwise the market is a guessing game and the payouts are arbitrary
Let traders see the resolution is honest	Otherwise the owner is an unaccountable judge
Not expose the person	They are a real student at a real university, and this is play money

The first two want the document public. The third wants it hidden. That tension is the whole design problem.

2. The first answer: redaction

The owner's initial choice was: publish the proof, but black out the private parts. Everything public, kept permanently as an audit trail, with automatic detection proposing what to hide and the owner confirming before anything shipped.

That is a reasonable answer and it is roughly what a newsroom or a court does. It was built:

- A private bucket for originals and a public one for redacted copies, with the privacy of each verified empirically rather than assumed.
- A renderer that **destroys pixels** rather than covering them. A black rectangle drawn over a PDF or layered on an image can be peeled off by anyone who downloads the file — that is how real redaction leaks happen. The renderer decoded the image, painted opaque black into the pixel data, and re-encoded it, so the published bytes had no relationship to the hidden content. Re-encoding also stripped EXIF metadata, including the GPS location a phone camera writes into a photo.
- A review screen showing the owner the whole original with suggested regions drawn on it, which they could adjust, add to or remove before publishing.

The safety argument was: the software proposes, a human decides, nothing publishes unseen.

2.1 Then it was measured

A fictional transcript was built carrying four things that must never be published — a student ID, a date of birth, a home address and a phone number — alongside the thing the document exists to prove, a GPA of 3.85.

The model was asked where the private information was. It answered:

- Student ID
- Date of Birth
- Home Address
- Phone Number

Four for four. It had read the document correctly and understood exactly which parts were sensitive.

Then the suggested regions were applied and the result was measured, pixel by pixel, across each line of the document:

Line	Should be hidden	Actually covered
Student ID	Yes	100%
Date of birth	Yes	35%
Home address	Yes	2%
Phone number	Yes	3%
Course grades	No — the point of the document	0%
Semester GPA	No — the point of the document	9%

Every suggested box sat roughly 85 pixels above the line it was meant to cover. The model's *reading*

was perfect and its *aim* was not.

2.2 Why that was worse than nothing

The naive conclusion is "the model needs to get better at coordinates." The real conclusion is harder.

A page showing four plausible black rectangles over a transcript looks finished. It invites the reviewer to glance, see that the software has clearly done its job, and press publish. The one control protecting the student — a human actually reading the page — is precisely the control that confident-looking output erodes.

A tool that fails visibly is safe. A tool that fails while looking successful is dangerous. Redaction with unreliable placement was the second kind.

And the consequence was unrecoverable. The design called for **public** and **permanent**. A missed redaction is a named student's home address, published forever, with no undo.

3. The second answer: publish a statement, not a document

The replacement inverts the problem.

Do not publish the document. Read it, keep it privately, and publish a sentence about it.

So instead of a redacted transcript, the goal page shows:

Official Ohio State University transcript shows Fall 2026 semester GPA of 3.85. —
Written by the owner after reading an original document supplied by Jordan. The app still holds that document; it is deliberately not shown.

Nothing confidential is published, and not because it was covered up. It was never carried across in the first place.

3.1 Why this is structurally safer

The redaction design managed a risk. This design removes it.

- **There is no public bucket.** There is no storage location an uploaded document could be published to, so no bug, no mistaken click and no future change can put one in front of the public.
- **No column on a public row names a stored file.** The `published_path` column was deleted along with the redaction design. The public projection carries the statement, the goal and the date, and nothing that could lead back to an object in storage.
- **The database refuses to record a violation.** A check constraint rejects any attempt to mark an uploaded document published without a verified statement attached. It is not a convention the code is trusted to follow; the database will not write the row.

A missed redaction was a leak. There is no equivalent failure here: the worst case is a badly worded statement, which is embarrassing and correctable, not a published transcript.

3.2 Why it also works better

The same measurement that killed redaction is the argument for this. The model read every value exactly right — 500-84-2291, 4417 Neil Avenue Apt 3B, Columbus OH 43201, 3.85. It was only bad at saying *where on the page* they sat.

Reading is the strength. Localization was the weakness. The new design only ever asks for the strength.

That also unlocked something the old design had to refuse: **PDFs are now accepted**. Under redaction, a PDF could not be taken, because the app could not rasterize and safely republish one — and accepting a format it could not publish would mean either refusing it later or publishing it unredacted. When nothing is republished, a PDF is just another thing to read. Since a transcript or an offer letter is usually a PDF, this matters.

4. How it works

4.1 The subject sends proof

From the moment a goal is approved until its proof deadline, the person the goal is about — and nobody else — can attach proof to it. A trader cannot. The owner cannot do it on their behalf.

Two kinds are accepted:

- **A document:** PDF, PNG, JPEG or WebP, up to 10 MB. Never published.
- **A link:** a plain <https://> address. Published exactly as submitted.

The distinction is honest and the form says so plainly. A URL cannot be summarised away — whoever opens it sees whatever is behind it. Submitting a link *is* a decision to publish it, so the form says so in bold, and the submitter is told to only paste something they are happy for strangers to see.

Nothing is visible to anyone but the sender and the owner until the owner acts.

4.2 The app reads it

When the owner opens a submission, the document is read against **that goal's own resolution terms**, not in the abstract. The model is asked for two separate things:

1. **Statements that could be published** — one short sentence each, carrying no private detail, plus what in the document supports it so the owner can check rather than trust.
2. **The private details present** — so the owner knows what must never be carried across.

Checked live against the fictional transcript, as both a PDF and an image:

	Result
GPA extracted correctly	Yes — "Fall 2026 semester GPA of 3.85"
Basis cited for each statement	Yes — "Document states 'Semester GPA: 3.85'"
Private details flagged	All four
Private details leaked into a statement	None

The two lists being separate is deliberate. The owner sees the proposed wording next to an explicit list of what must not appear in it.

4.3 The owner decides

The review screen shows the original document, the suggested wording, and the list of private details. The owner writes or edits the statement and publishes it, or rejects the submission with a private note.

The screen is built to make the owner look rather than click:

- The suggestions are labelled as suggestions. Nothing claims the document was "checked" or is "clean."
- An empty list of findings says so explicitly: the model found nothing, which is not the same as nothing being there.
- The statement field starts empty. Accepting a proposal is a deliberate click, not a default.

4.4 What the public sees

The goal page carries the statements under an attestation: each was written by the owner after reading an original the app still holds, deliberately not shown because it carries personal details that are nobody else's business.

That attestation is the contract. It tells a trader exactly what they are relying on — the owner's word, backed by a document the owner has and they do not.

5. What it costs

This is a real trade and it should be stated plainly.

Traders can no longer check the resolution themselves. The original motivation for publishing proof was that anyone with money on a market could audit the ruling. They cannot now. They see the owner's statement and the owner's reasoning, not the evidence.

Transparency moved from *"look at it yourself"* to *"the owner says this, and their record is what backs it."*

For an app among friends at one university, with one known owner whose judgment is the product, that is a reasonable trade. It would be a worse one at scale, with an owner nobody knows. Three things partly offset it:

- The owner's written explanation is public and permanent.
- Every ruling can be objected to within 24 hours before payout becomes final.
- The document is retained, so a dispute can be adjudicated even if not publicly.

What is still open:

- **Correcting a published statement.** Not decided. A wrong statement currently stands.
- **Deleting a single item without leaving.** Not decided.
- **Deletion on withdrawal.** Decided, unbuilt: withdrawing removes that person's documents and leaves a tombstone plus the ruling record. Payouts are unaffected, since settlement is already final and the ledger is untouched.

6. What holds it together

The safety properties, and how each is enforced rather than intended:

Property	Enforced by
Only the subject can submit proof	Identity checked before anything else, so a stranger learns nothing about the goal's state
Originals are unreadable without authorization	A private bucket, verified empirically: the browser key was blocked, an unauthenticated request returned 400, only the server key and a 5-minute signed link succeeded
No document can become public	No public bucket exists; no public column names a stored file
Nothing publishes without a human	A database check requiring an attributable review and a statement
A model outage cannot block review	Every failure returns an empty result with a reason; the owner writes the statement themselves
Provider errors never reach a browser	Only the app's own error messages are returned; storage, database and provider messages are swallowed

Verified: 371 tests pass, 17 skipped (hosted-only). Typecheck, lint and production build clean. Storage privacy re-verified by probe. Extraction checked live against a fictional transcript in both PDF and image form.

7. The general lesson

The useful thing here is not the feature. It is the sequence.

An approach was chosen, built completely and carefully — real pixel destruction, metadata stripping,

verified bucket privacy, an approval gate — and then **measured against a case designed to expose it**. The measurement showed it failing in the specific way that mattered, while looking like it worked.

Had it shipped, it would have shipped confidently. The tests passed. The renderer did exactly what it claimed. The privacy checks were real. Everything was correct except the one number nobody had looked at: how much of the address was actually covered.

The replacement is simpler, needs no image processing at all, accepts more file types, and cannot fail the same way — because it removes the failure mode instead of managing it.

The safest design is not the one with the best safeguards. It is the one where the dangerous thing never happens.